

Übung 2

1 Game Engine und Spielidee

Aufbauend auf Übung 1 wurde für die zweite Übung ebenfalls die Unity Engine verwendet. Der grundlegende Aufbau der Szene wurde für diese Aufgabe beibehalten. Ein Labyrinth wird auf Grundlage einer Grafik erzeugt und diverse Assets platziert. Im Gegensatz zu der ersten Übung wurde der goldene GoalBall entfernt und durch einen Nicht-Spieler-Charakter (NPC) ersetzt. Dieser NPC wird über ein Large Language Model (LLM) gesteuert. Hierzu werden in regelmäßigen Abständen Statusinformationen über die Spielumgebung an das LLM gesendet und dieses soll den NPC steuern. Sobald der Spieler den NPC berührt, wird die Spielszene zurückgesetzt. Im Vergleich zu der ersten Übung wurden einige Grafikeinstellungen und das Raytracing reduziert, um eine flüssigere Anzeige zu ermöglichen. Die verwendete Hardware entspricht dem ersten Übungsblatt. Die Skripte wurden unter Verwendung von ChatGPT erstellt ¹.

2 NPC Model

Das Charaktermodell des NPC wurde mit Blender² erstellt. Zur Orientierung diente ein Youtube Tutorial, welches die grundlegende Vorgehensweise erläutert [Carl24a]. Für das Charaktermodell wurden anschließend ein Skelett (Rig) sowie mehrere Animationen erzeugt, die Grundlagen hierzu sind ebenfalls aus einem Tutorial entnommen [Carl24b, Carl21]. Der Charakter wurde mitsamt den Animationen idle, gehen, winken und ducken aus Blender exportiert und in Unity importiert.

In Unity wurde eine Entität erstellt, die den NPC und dessen Logik beinhaltet. Die Struktur ist in Abbildung 1 dargestellt.

Diese Entität besitzt zwei Collider, einen für die Physik und einen für den Zieltrigger. Der Physik-Collider sorgt dafür, dass der NPC nicht durch Wände laufen oder den Boden fallen kann. Mit Hilfe

¹<https://chatgpt.com/>

²<https://www.blender.org/download/>

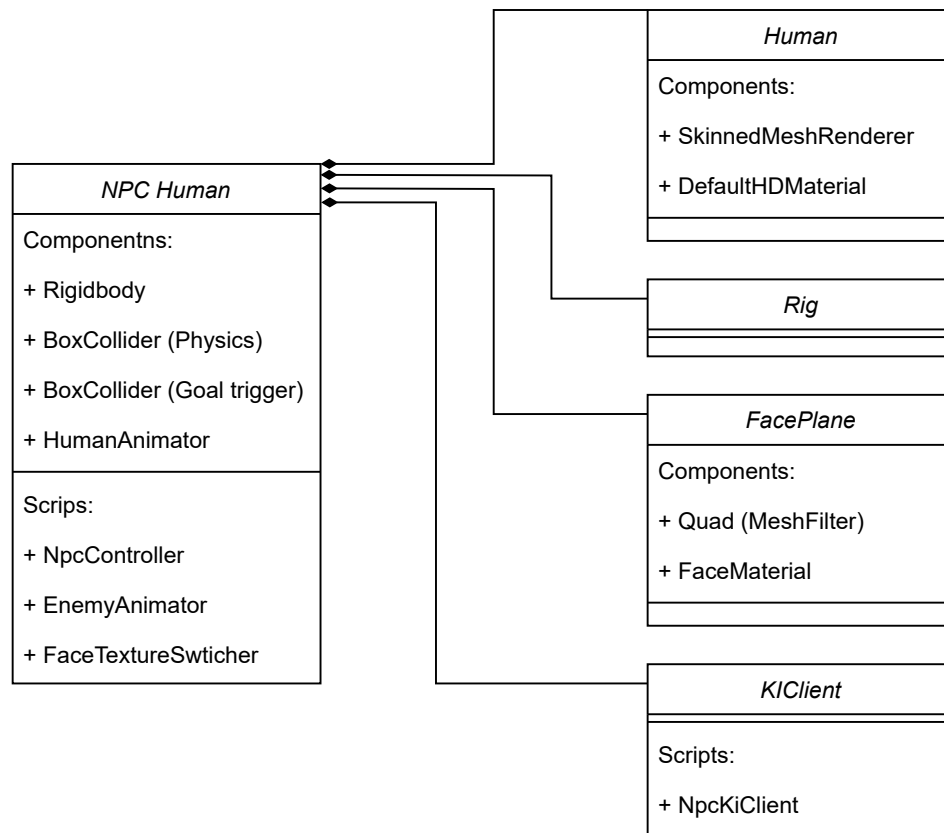


Abbildung 1: Struktur des NPC

des Zieltriggers soll die Szene neu gestartet werden, sobald eine Kollision mit dem Spieler erkannt wurde. Der HumanAnimator ist für die Animation zuständig, wobei die Funktionen zum Ändern der Animationen im Skript **EnemyAnimator** implementiert sind. Der NPC soll je nach Stimmung seine Farbe und einen Gesichtsausdruck ändern. Der Gesichtsausdruck wird über die FacePlane realisiert, die unmittelbar in Kopfhöhe des Charaktermodells platziert ist. Die unterschiedlichen Texturen befinden im Ordner **Resources/Faces**. Die unterschiedlichen Gesichtsausdrücke wurden nicht in Github veröffentlicht. Das fertige Modell ist in [Abbildung 2](#) dargestellt.

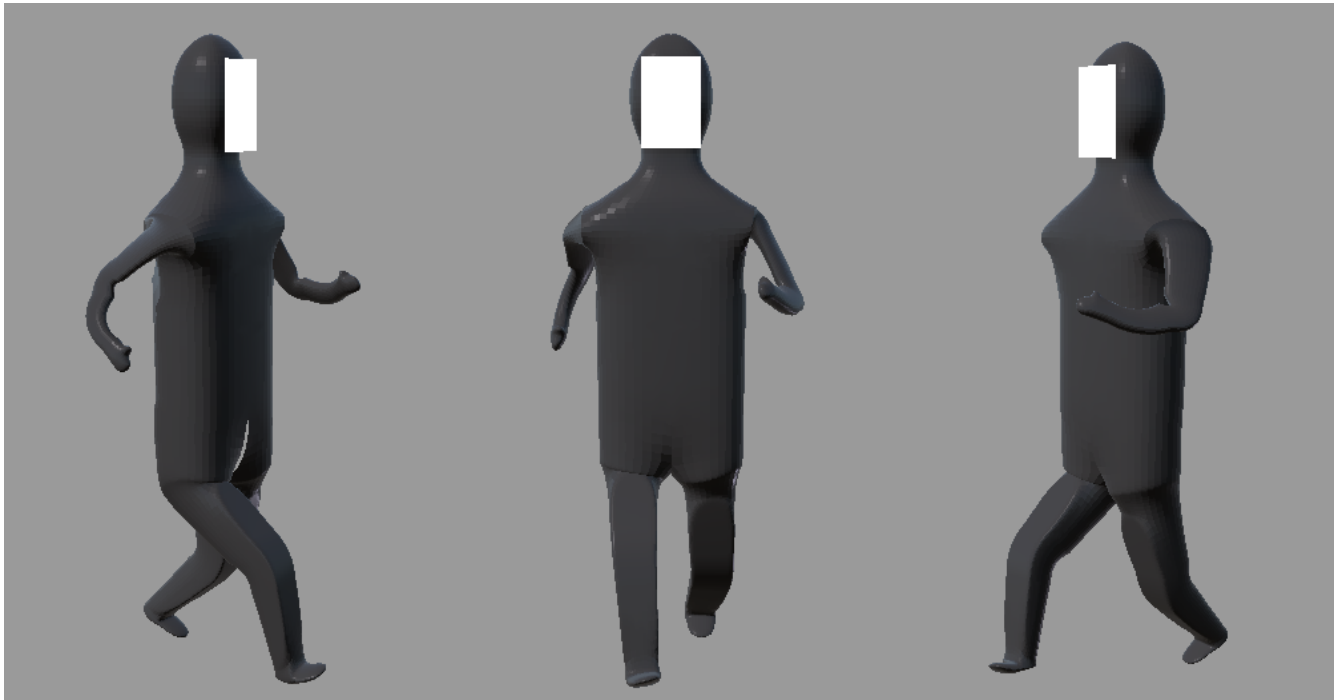


Abbildung 2: Charaktermodell des NPC

Insgesamt sind vier verschiedene Emotionen vorgesehen. Je nach Stimmung ändert sich die Farbe des Charakters wie folgt:

- angry -> Rot
- scared -> Magenta
- happy -> Blau
- calm -> Grün

3 Logik des NPC

Die Logik des NPC ist grundlegend über die Skripte `NpcController` sowie `NpcKiClient` realisiert. Die Funktionsweise der Skripte wird in den nachfolgenden Abschnitten erläutert.

3.1 NpcKiClient

In diesem Skript ist die Logik zur Kommunikation mit dem LLM über eine API realisiert. Die Einstellungen der API sind hier hinterlegt und die Nachricht wird im JSON Format zusammengesetzt. Hierbei besteht jede Nachricht immer aus einer Systemanweisung, die das Spiel erklärt sowie einem userPrompt, der aktuelle Statusinformationen des Spiels enthält. Das LLM erhält die Anweisung den NPC zu steuern, indem eine Aktion, eine Richtung und eine Emotion zurückgegeben ausgegeben werden soll. Der Aufbau des userPrompts wird im folgenden Abschnitt beschrieben. Die zurückgegebene Antwort wird in der Funktion `ParseFlexible` verarbeitet. Für den Fall, dass keine gültiges Ergebnis festgestellt wird, ist eine Standardanweisung hinterlegt.

3.2 NpcController

Dieses Skript steuert den NPC, indem alle 5 Sekunden eine Anfrage an das LLM gesendet und verarbeitet werden. Die an das LLM übermittelten Statusinformationen beinhalten

- letzte NPC Emotion (nach der letzten Anfrage)
- letzte NPC Aktion (nach der letzten Anfrage)
- letzter Abstand zum Spieler (Zeitpunkt der letzten Anfrage)
- aktueller Abstand zum Spieler
- Winkel zum Spieler (0 = Nord)
- Spieler in Sicht (mittels Raycasting)

In Abhängigkeit der Antwort des LLM ändert das Skript die aktuelle Aktion, Emotion und Richtung. Zudem wird die Szene bei Kollision mit dem Spieler neu erstellt.

4 Reflexion

In diesem Übungsblatt wurde ein NPC in das Labyrinth integriert, welcher von einem LLM gesteuert wird. Hierzu wurde ein Charakter mit zugehörigen Animationen von Grund auf in Blender erstellt. Da keine Vorkenntnisse im Umgang mit dieser Software vorhanden waren, konnte grundlegende Kompetenzen in diesem Bereich erlangt werden. Dies erforderte jedoch eine entsprechende Einarbeitungszeit. Es bleibt festzustellen, dass die Bedienung von Blender nicht sehr intuitiv ist und im Rahmen der Bearbeitung auch eine Komplexen oder Detailreichen Ergebnisse erzielt werden konnten. Als LLM wurde das von der Universität bereitgestellte Modell verwendet. Sobald die grundlegende Systemanweisung erstellt war, lieferte das LLM auch zuverlässig Anweisungen im geforderten Format zurück. Hierbei ist jedoch festzustellen, dass das LLM eher eine eintönige Auswahl an Emotionen und Aktionen zurücklieferte. Hierbei passte sich das Ergebnis auch stets den Gegebenheiten der Umgebung an und lieferte konstant Ausgaben im geforderten Format. Der NPC scheint jedoch entgegen der Anweisung den Spieler eher zu verfolgen anstatt vor ihm zu flüchten, was der Anweisung entsprechen würde. Die eingeschränkte Funktionsweise des LLM dürfte hierbei jedoch auf die geringe Komplexität des verwendeten Modells zurückzuführen sein. Dies stellt jedoch einen Kompromiss aus Leistung und Antwortzeit dar, was für Echtzeitspiele essentiell sein müsste. Dass es grundsätzlich möglich ist, einen NPC mit einem LLM zu steuern, kann hier angenommen werden. Wobei der Grad der Geeignetheit von der Art des Spiels abhängt. In diesem experimentellen Setting bei einer Anfrage alle 5 Sekunden ist das Spiel für den Spieler nur in geringem Maße fordernd. Wie sich ein größeres LLM, kürzere Abfragezeiten, andere Statusinformationen oder die eventuelle Verwendung einer Nachrichtenhistorie als Eingabe auswirken, kann in künftigen Arbeiten untersucht werden.

Literatur

- [Carl21] J. Carlino. Easy Walk Cycle, Character Animation with Blender. <https://www.youtube.com/watch?v=nRtT7Gr6S2o>, 2021. Stand: 23.11.2021, Aufruf am: 06.07.2025.
- [Carl24a] J. Carlino. Character modeling for beginners - Blender. <https://www.youtube.com/watch?v=O6HQhs-gk50>, 2024. Stand: 29.04.2024, Aufruf am: 06.07.2025.
- [Carl24b] J. Carlino. Rigging for beginners - Blender tutorial. https://www.youtube.com/watch?v=m-Obo_nC3SMt, 2024. Stand: 30.05.2024, Aufruf am: 06.07.2025.